Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №6

з дисципліни „ Професійний Python”

на тему „ Автоматизоване тестування Python-програм.”

Виконала:
студентка 3 курсу
групи ФеП-32
Клімащук Вероніка Ігорівна
Перевірила:
Доц. Куньо Іван Михайлович

Львів – 2026

1. Тема, мета й тестований проєкт

Мета роботи — сформувати навички автоматизованого тестування Python-програм через pytest: розділяти unit та integration tests, готувати fixtures, параметризувати сценарії, перевіряти помилки, ізолювати файлові операції, підміняти зовнішні залежності та аналізувати statement і branch coverage.

Варіант №9 узгоджений із предметною областю ЛР №3 і №5 — система обліку спортивних результатів. Тестований пакет імпортує CSV, валідує athlete/sport/result/category/timestamp за конфігурацією, рахує статистику, фільтрує та ранжує результати, експортує JSON і звіт помилок. Для ЛР №6 додані тестові порти repository/notifier і асинхронний зовнішній provider, щоб перевірити взаємодії через Mock та AsyncMock.

2. Структура production code і test suite

```
SportsResultsLab06/
├── pyproject.toml
├── README.md
├── src/sports_streaming/
│   ├── models.py, validators.py, config.py, readers.py
│   ├── services.py, analytics.py, filters.py, batches.py
│   ├── exporters.py, logging_config.py, main.py
│   ├── repositories.py, notifications.py, application.py
│   └── async_ranking.py
├── tests/
│   ├── conftest.py
│   ├── unit/
│   │   ├── test_models_validators.py
│   │   ├── test_analytics.py
│   │   ├── test_io_config_services.py
│   │   └── test_application_async.py
│   └── integration/
│       ├── test_sports_pipeline.py
│       └── test_main_failures.py
└── report/
    ├── test_results.txt
    └── coverage/index.html
```

Pytest конфігурація лежить у pyproject.toml: testpaths, pythonpath, strict markers, asyncio auto mode, branch=True та fail_under=80. Звіт HTML з coverage.py збережений у report/coverage/.

3. Test matrix

Рівень / зона	Що перевіряється	Основні сценарії
Unit: domain і validation	SportsResult та parse_record	нормалізація; мінімум/максимум; некоректний тип, знак, діапазон, поле й час; chaining
Unit: analytics	ranking, filters, averages, pairwise, batches	порожній набір; top-N; сумісність виду спорту/категорії; останній неповний batch
Unit: files/config	CSV, YAML, JSON, error CSV	missing headers/file; зайві стовпці; malformed YAML; env override; atomic replace failure
Unit: interactions	ResultApplication та provider	Mock return_value; side_effect failure; notifier не викликається після failure; AsyncMock awaited

Integration	повний import/export і CLI	CSV→validation→JSON/error CSV; strict збереження попереднього output; CLI limit та failure; async ranking

У наборі понад 15 unit tests і не менше 3 integration tests. Фактичний запуск зібрав 61 test case: 53 unit та 8 integration, включно з варіантами параметризованих тестів.

4. Fixtures і ізоляція

conftest.py надає чотири спільні fixtures: result_factory створює доменний запис із потрібними значеннями; app_config готує незалежну конфігурацію з tmp_path; configured_csv створює приклад CSV; result_records формує набір для аналітики. Repository fixture використовує yield і очищає стан після тесту.

```
@pytest.fixture
def app_config(tmp_path):
    source = tmp_path / "input.csv"
    source.write_text("athlete,sport,result,category,timestamp\n", encoding="utf-8")
    yield AppConfig(input_path=source, output_path=tmp_path / "out.json", ...)
```

tmp_path дає кожному тесту окрему тимчасову директорію. Тести не пишуть у production data/output і не потребують мережі.

5. Parametrization, boundaries та винятки

```
@pytest.mark.parametrize("value", ["0", "100000"], ids=["minimum", "maximum"])
def test_result_boundaries_are_valid(value, app_config):
    parsed = parse_record({**BASE, "result": value}, line_number=2, config=app_config)
    assert parsed.result == float(value)
with pytest.raises(RecordValidationError) as exc:
    parse_record({**BASE, "result": "wrong"}, line_number=4, config=app_config)
assert isinstance(exc.value.__cause__, ValueError)
```

Параметризовані перевірки окремо охоплюють нижню й верхню межі, а також athlete, sport, category, numeric result, NaN, negative result, надмірне значення й timestamp. pytest.raises перевіряє тип і текст помилки; exc.value дає доступ до field, line_number та __cause__.

6. Float, monkeypatch та конфігурація

```
assert mean_result(result_records[:2]) == pytest.approx(12.7)
monkeypatch.setenv("SPORTS_RESULTS_MODE", "strict")
assert load_config(config_path).mode == "strict"
```

pytest.approx використовується для чисел із плаваючою комою. monkeypatch тимчасово задає SPORTS_RESULTS_MODE; pytest сам відновлює початкове середовище після тесту. YAML читається з tmp_path, тому кожен сценарій конфігурації незалежний.

7. Mock, side_effect і перевірка interactions

```
repository = Mock(spec=ResultRepository)
repository.save.return_value = "R-44"
notifier = Mock(spec=ResultNotifier)
service = ResultApplication(repository, notifier)
assert service.register(result) == "R-44"
repository.save.assert_called_once_with(result)
notifier.notify.assert_called_once_with("Sports result R-44 recorded")
repository.save.side_effect = OSError("storage unavailable")
```

```
with pytest.raises(OSError): service.register(result)
notifier.notify.assert_not_called()
```

Mock замінює лише межу до repository/notifier. return_value перевіряє передачу ідентифікатора, side_effect моделює збій сховища. Interaction assertions підтверджують один виклик save, правильне повідомлення та відсутність сповіщення після невдалого запису.

8. AsyncMock і integration tests

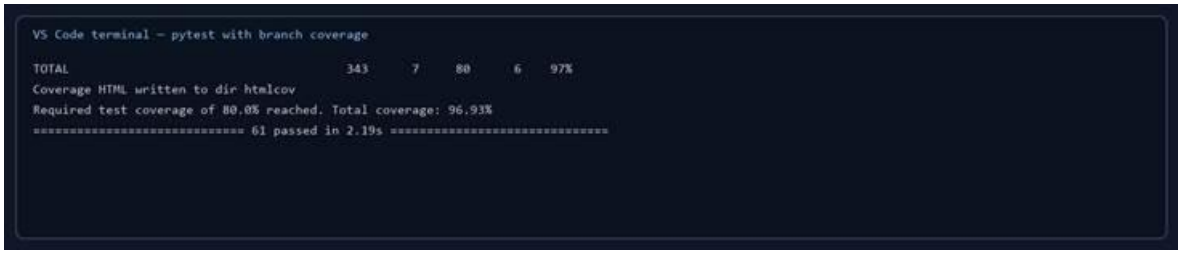
```
provider = AsyncMock()
provider.fetch_results.return_value = [slow_result, fast_result]
ranking = await fetch_ranking(provider, "running", top_n=1)
assert ranking == [fast_result]
provider.fetch_results.assert_awaited_once_with("running")
```

AsyncMock ізолює зовнішнє асинхронне джерело результатів і підтримує як успіх, так і TimeoutError через side_effect. Чотири integration-тести проганяють імпорт та експорт, перевіряють збереження старого JSON при strict-помилці, рейтинг від зовнішнього provider і командний main flow з config/log/output.

9. Результати запуску

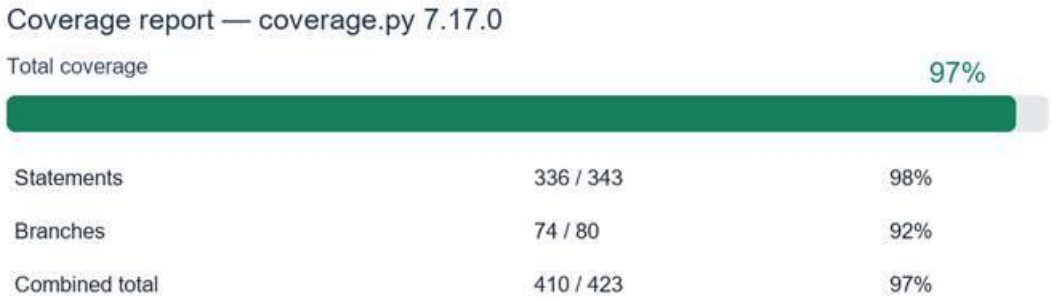
```
python -m pytest -v --cov=sports_streaming --cov-branch \
  --cov-report=term-missing --cov-report=html
```

Повний запуск на Python 3.12.2 із pytest 9.1.1 завершився успішно: 61 passed, 0 failed. Для coverage увімкнуто branch tracing.



Метрика coverage.py	Покрито	Усього	Показник
Statements	336	343	98%
Branches	74	80	92%
Разом	410	423	97%

HTML summary показує округлене загальне покриття 97%; точне combined total у консолі — 96,93%. Branch coverage становить 74 із 80 переходів (92%), вище заданого мінімуму. Повний файл index.html та його модульні сторінки доступні в report/coverage/.



HTML-фрагмент звіту

```
<h1>Coverage report: <span class="pc_cov">97%</span></h1>
<tr class="total">
  <td>Total</td><td data-ratio="336 343">98%</td>
  <td>343</td><td>7</td>
  <td data-ratio="74 80">92%</td><td>80</td><td>6</td>
  <td data-ratio="410 423">97%</td>
</tr>
```

10. Аналіз непокритих paths і знайдені дефекти

Coverage позначив кілька гілок перевірки конфігурації, які не є основними happy path: YAML із коректним синтаксисом, але не-мапою; відсутні/неправильні ключі; порожній перелік дозволених значень; невідповідність наборів правил; недозволений рівень логування. Їх можна покрити окремими параметризованими сценаріями, якщо потрібне покриття вище за встановлений поріг. Одна пропущена лінія main.py — стандартна умова запуску через __main__, не бізнес-гілка.

У фінальному запуску дефектів production-коду не виявлено. Тести підтвердили контрольований збій strict імпорту, chaining першопричини, відсутність залишкового тимчасового файла та збереження попереднього результату. Перші невдалі прогони під час налаштування виявили особливість обмеженої тимчасової директорії Windows і надто суворе очікування, що error CSV повторить сире значення; tests переналаштовано на робочу ізольовану tmp_path і контрактне повідомлення помилки.

11. Відповіді на контрольні запитання

1. Що таке automated test? Код, який автоматично запускає сценарій і перевіряє очікуваний результат.

2. Які основні переваги автоматизованого тестування? Воно знаходить помилки раніше, попереджає регресії та фіксує поведінку.

3. Що таке regression? Повернення раніше виправленої помилки або поява нової після змін.

4. Що таке unit test? Перевірка малої одиниці поведінки ізольовано.

5. Що таке integration test? Перевірка взаємодії кількох компонентів системи.

6. Чим integration test відрізняється від unit test? Integration перевіряє компоненти разом, unit — окремо.

7. Що показує test pyramid? Бажане співвідношення великої кількості unit і меншої кількості дорожчих integration/E2E тестів.

8. Чому unit tests зазвичай більше? Вони швидші, точніше локалізують проблему та дешевші в підтримці.

9. Що таке pytest? Python-фреймворк для виявлення й запуску тестів та перевірки assertions.

10. Як pytest знаходить tests? За типовими іменами test_*.py, *_test.py та функціями test_*.

11. Якою командою запускається pytest? Командою python -m pytest із кореня проєкту.

12. Для чого pytest -v? Для докладного переліку кожного тесту.

13. Для чого pytest -x? Щоб зупинитися після першої помилки.

14. Що означає Arrange–Act–Assert? Підготувати стан, виконати дію, перевірити результат.

15. Для чого використовується assert? Для явної перевірки умови, яку має гарантувати тест.

16. Що таке assertion failure? Невиконання очікуваної умови тесту.

17. Чому floats не завжди слід порівнювати через ==? Через двійкове округлення десяткові дроби можуть зберігатися неточно.

18. Для чого pytest.approx? Для порівняння чисел із допустимою похибкою.

19. Для чого pytest.raises? Для перевірки, що код піднімає очікуваний виняток.

20. Як перевірити exception message? Передати match у pytest.raises або перевірити str(exc_info.value).

21. Як отримати exception object через exc_info? Використати pytest.raises як контекстний менеджер і звернутися до exc_info.value.

22. Що таке boundary testing? Перевірка значень на межі допустимого діапазону та поруч із нею.

23. Які boundaries потрібно тестувати для діапазону 0–100? 0 і 100 як дозволені, а також значення трохи нижче та вище.

24. Що таке parametrization? Повторний запуск однієї тестової функції з різними наборами аргументів.

25. Для чого pytest.mark.parametrize? Щоб описати дані сценаріїв декларативно й уникнути дублювання тестів.

26. Що дають ids параметризованих tests? Зрозумілі назви кожного параметричного випадку у звіті.

27. Що таке fixture? Функція підготовки даних або ресурсів для тесту.

28. Для чого використовуються fixtures? Для повторного використання контрольованого setup і teardown.

29. Що означає fixture setup? Підготовку стану до виконання тесту.

30. Що означає fixture teardown? Очищення ресурсів після завершення тесту.

31. Для чого yield у fixture? Віддає тесту ресурс, після чого виконує код очищення.

32. Які fixture scopes підтримує pytest? function, class, module, package та session.

33. Який scope є default? function.

34. Для чого conftest.py? Для спільних fixtures і pytest hooks у каталозі тестів.

35. Чому fixtures із conftest.py не імпортують вручну? Pytest автоматично знаходить їх у відповідному дереві тестів.

36. Що таке tmp_path? Унікальна тимчасова директорія pathlib.Path для одного тесту.

37. Чому tmp_path корисний? Ізолює файлові операції й не залишає тестові файли у проєкті.

38. Що таке test isolation? Незалежність тесту від стану інших тестів і зовнішнього середовища.

39. Для чого monkeypatch? Тимчасово підміняти атрибути, функції, змінні середовища й робочу директорію.

40. Як monkeypatch змінює environment variable? Викликом monkeypatch.setenv("KEY", "value").

41. Як видалити environment variable у test? Викликом monkeypatch.delenv("KEY", raising=False).

42. Як підмінити function через monkeypatch? Викликати monkeypatch.setattr(module, "function", fake).

43. Що таке test double? Загальна назва об’єкта, що замінює реальну залежність у тесті.

44. Що таке Dummy? Об’єкт-заповнювач, який передають, але не використовують.

45. Що таке Stub? Заміна, яка повертає заздалегідь задану відповідь.

46. Що таке Fake? Спрощена робоча реалізація, наприклад in-memory repository.

47. Що таке Mock? Заміна, що зберігає виклики й дає перевірити взаємодію.

48. Що таке Spy? Об’єкт, який фіксує виклики реальної реалізації.

49. Для чого Mock? Ізолювати системну межу й перевірити передані аргументи та виклики.

50. Що таке spec? Обмеження mock атрибутами реального інтерфейсу.

51. Для чого return_value? Задати результат, який поверне mock-функція.

52. Для чого side_effect? Змусити mock підняти виняток або повертати послідовні результати.

53. Як перевірити, що method викликано один раз? Використати mock.method.assert_called_once().

54. Як перевірити parameters виклику? Використати assert_called_once_with(expected_args).

55. Як перевірити, що method не викликався? Використати mock.method.assert_not_called().

56. Що таке state testing? Перевірка стану об’єкта після операції.

57. Що таке interaction testing? Перевірка викликів і передачі даних між залежностями.

58. Коли interaction testing виправдане? Коли контракт вимагає конкретної взаємодії або заборони зайвих викликів.

59. Що таке patch? Тимчасова заміна об’єкта чи функції під час тесту.

60. Чому patch потрібно виконувати там, де name використовується? Тест має замінити посилання, яке реально шукає код під тестом.

61. Що таке AsyncMock? Mock для async-функцій, який можна await-ити й перевіряти awaited виклики.

62. Як перевірити awaited call? Використати assert_awaited_once_with(expected_args).

63. Що таке flaky test? Тест, що нестабільно проходить або падає без зміни коду.

64. Чому flaky tests небезпечні? Вони знижують довіру до CI та маскують справжні регресії.

65. Що таке deterministic test? Тест із повторюваним результатом за однакових умов.

66. Чому unit test не повинен залежати від Internet? Мережа додає повільність, нестабільність і зовнішні причини збоїв.

67. Коли в integration test доцільний real filesystem? Коли перевіряється реальна взаємодія компонентів із файлами у tmp_path.

68. Чому не потрібно mock-ати все? Інакше тест перевірить домовленості mock-ів замість поведінки системи.

69. Що таке over-mocking? Надмірна заміна реальних компонентів mock-ами без перевірки реального результату.

70. Що таке coverage? Вимірювання частини коду, яку виконали тести.

71. Що таке statement coverage? Частка виконаних інструкцій програми.

72. Що таке branch coverage? Частка перевірених переходів умов, наприклад true та false.

73. Чим вони відрізняються? Statement показує виконання рядків, branch — перевірку альтернативних напрямків.

74. Як отримати coverage через pytest-cov? Запустити pytest з прапорцем --cov=package.

75. Для чого --cov-report=term-missing? Показати підсумок у терміналі й номери незапущених рядків.

76. Для чого HTML coverage report? Для інтерактивного огляду покриття файлів, рядків і гілок.

77. Чи гарантує 100% coverage відсутність bugs? Ні, код міг виконатися без якісних assertions або неповного набору сценаріїв.

78. Чому meaningful assertions важливіші за coverage percentage? Вони перевіряють правильність результату, а не лише факт виконання коду.

79. Що таке coverage threshold? Мінімальний поріг покриття, нижче якого запуск вважається невдалим.

80. Чому tests повинні бути незалежними один від одного? Щоб їхній результат не залежав від порядку виконання чи чужого стану.

12. Висновок

Створено автоматизований test suite для системи спортивних результатів, який охоплює доменну валідацію, аналітику, CSV/YAML/JSON, атомарний експорт, конфігурацію, CLI, синхронні й асинхронні залежності. Тести організовано за unit та integration рівнями; використовуються fixtures, parametrization, tmp_path, monkeypatch, pytest.raises, pytest.approx, Mock, AsyncMock і branch coverage.

У фінальному запуску пройшли всі 61 тест. Statement coverage становить 98%, branch coverage — 92%, combined total — 97%, що перевищує рекомендований поріг 80%.